VaultX

Feature Enhancement Guide

12 Features · Step-by-Step Code for Every Feature

From beginner-friendly to truly extraordinary

VIT Pune | CSAIML-A | Group 3 | AY 2025-26 Sem 2

Overview — 12 Features at a Glance

The table below lists all 12 features in priority order. Start with the ones marked Easy and work toward Advanced. Each feature has its own section with a full explanation, exact code, and how to explain it to faculty.

Feature	Difficulty	Wow Factor	Est. Time
1. Two-Factor Authentication (TOTP)	Medium	⋆⋆⋆⋆⋆	2–3 hours
2. Auto-Lock After Inactivity	Easy	⋆⋆⋆⋆	30 minutes
3. Password History & Versioning	Easy-Medium	⋆⋆⋆⋆	1 hour
4. Vault Health Dashboard	Medium	⋆⋆⋆⋆⋆	2 hours
5. Browser Extension (Auto-Fill)	Hard	⋆⋆⋆⋆⋆	4–6 hours
6. Argon2id (Upgrade from PBKDF2)	Easy	⋆⋆⋆⋆⋆	30 minutes
7. Secure Notes (Encrypted Markdown)	Easy	⋆⋆⋆	45 minutes
8. Import from Chrome / Firefox	Medium	⋆⋆⋆⋆	1.5 hours
9. Login Attempt Limiting	Easy	⋆⋆⋆⋆	30 minutes
10. Password Expiry Reminders	Easy	⋆⋆⋆	45 minutes
11. Vault Search with Fuzzy Matching	Easy	⋆⋆⋆	30 minutes
12. Desktop App with CustomTkinter	Medium	⋆⋆⋆⋆	3–4 hours

RECOMMENDED ORDER FOR PRESENTATION IMPACT

Start with Feature 6 (Argon2id) — it takes 30 minutes and immediately shows you know better algorithms than PBKDF2.
Add Feature 9 (login limiting) — simple and directly answers 'is it secure against brute force?'
Add Feature 2 (auto-lock) — shows awareness of physical security, not just cryptographic security.
Then add Feature 1 (TOTP 2FA) — this is the single most impressive feature you can add.
Finally add Feature 4 (health dashboard) — visually stunning, faculty can see it instantly.

FEATURE 01 **Two-Factor Authentication (TOTP)**

Difficulty: Medium Impact: MAXIMUM Est. Time: 2–3 hours

What It Is

TOTP stands for Time-based One-Time Password. It is the 6-digit code that Google Authenticator and Microsoft Authenticator generate every 30 seconds. Adding 2FA to VaultX means that even if someone steals your master password, they still cannot open the vault without also having your phone.

This is feature is used by every major platform in the world — Gmail, GitHub, Instagram, banks. Adding it to a 1st-year project immediately signals to faculty that you understand real-world security architecture.

How It Works (Explain This to Faculty)

When 2FA is enabled, a random 160-bit secret key is generated and stored (encrypted) in the vault. This key is shared with the authenticator app by scanning a QR code. Every 30 seconds, both the app and VaultX independently apply HMAC-SHA1 to the secret key combined with the current Unix timestamp divided by 30. Because both sides use the same formula, they always produce the same 6-digit code. An attacker without the secret key cannot produce a valid code.

Step-by-Step Implementation

Step 1 — Install the pyotp library

```
pip install pyotp qrcode pillow
```

pyotp handles all TOTP math. qrcode generates the QR image. pillow (PIL) handles image rendering.

Step 2 — Create totp.py

Create a new file called totp.py in your vaultx folder and add the following code exactly:

```
import pyotp
import qrcode
import io, base64, json, os
from crypto import encrypt, decrypt
```

```
TOTP_FILE = "totp_config.json"

def generate_totp_secret() -> str:
    """Generate a new 160-bit base32 TOTP secret."""
    return pyotp.random_base32()   # e.g. "JBSWY3DPEHPK3PXP"

def save_totp_secret(key: bytes, secret: str):
    """Encrypt the TOTP secret before saving to disk."""
    enc = encrypt(key, secret)
    with open(TOTP_FILE, "w") as f:
        json.dump(enc, f)

def load_totp_secret(key: bytes) -> str | None:
    """Load and decrypt the TOTP secret."""
    if not os.path.exists(TOTP_FILE):
        return None
    with open(TOTP_FILE) as f:
        enc = json.load(f)
    return decrypt(key, enc["nonce"], enc["ciphertext"])

def verify_totp(key: bytes, code: str) -> bool:
    """Verify a 6-digit TOTP code. valid_window=1 allows 30s clock drift."""
    secret = load_totp_secret(key)
    if not secret:
        return False
    totp = pyotp.TOTP(secret)
    return totp.verify(code, valid_window=1)

def get_qr_base64(key: bytes, account_name: str = "VaultX") -> str:
    """Generate QR code as a base64 image for display in browser."""
    secret = load_totp_secret(key)
    uri    = pyotp.totp.TOTP(secret).provisioning_uri(
                 name=account name, issuer name="VaultX")
    img    = qrcode.make(uri)
    buf    = io.BytesIO()
    img.save(buf, format="PNG")
    return base64.b64encode(buf.getvalue()).decode()
```

Step 3 — Add 2FA routes to app.py

Add these new routes to app.py after your existing routes:

```
from totp import generate_totp_secret, save_totp_secret, verify_totp,
get_qr_base64

# ── Enable 2FA ──────────────────────────────────
@app.route("/settings/2fa/enable", methods=["GET", "POST"])
def enable_2fa():
    if "aes_key" not in session:
        return redirect(url for("login"))
    key = bytes.fromhex(session["aes_key"])

    if request.method == "POST":
        code = request.form.get("code", "")
        if verify_totp(key, code):
            session["2fa_verified"] = True
            flash("2FA is now active!", "success")
```

```
            return redirect(url_for("vault"))
        flash("Invalid code. Try again.", "error")

    # GET: generate secret and show QR
    secret = generate_totp_secret()
    save_totp_secret(key, secret)
    qr = get_qr_base64(key)
    return render_template("setup_2fa.html", qr=qr)

# ── 2FA Verify step (after password login) ──────
@app.route("/verify-2fa", methods=["GET", "POST"])
def verify_2fa():
    if "pending_key" not in session:
        return redirect(url for("login"))

    if request.method == "POST":
        code = request.form.get("code", "")
        key  = bytes.fromhex(session["pending_key"])
        if verify_totp(key, code):
            session["aes key"] = session.pop("pending key")
            return redirect(url_for("vault"))
        flash("Invalid 2FA code.", "error")

    return render template("verify 2fa.html")
```

Step 4 — Modify the login route

In app.py, change the login route so that if 2FA is enabled, it does not immediately grant access — instead it puts the key in 'pending' and redirects to the 2FA verification step:

```
# Inside the login route, replace the success block:
if verify_master_password(password):
    key = derive_key(password)
    import os as _os
    if _os.path.exists("totp_config.json"):
        # 2FA is enabled — hold the key in pending
        session["pending_key"] = key.hex()
        return redirect(url_for("verify_2fa"))
    else:
        # No 2FA — grant access directly
        session["aes_key"] = key.hex()
        return redirect(url_for("vault"))
```

WHAT TO SAY TO FACULTY

"We implemented TOTP two-factor authentication using the RFC 6238 standard — the same standard used by Google Authenticator. Even if someone steals the master password, they cannot log in without the 6-digit rotating code from the phone. The TOTP secret itself is encrypted with AES-256-GCM before being stored on disk, so even the 2FA seed is protected."

FEATURE 02 **Auto-Lock After Inactivity**

Difficulty: Easy Impact: HIGH Est. Time: 30 minutes

What It Is

If you leave the vault open and walk away, someone could access your passwords. Auto-lock detects when the user has been inactive for a set time (e.g. 5 minutes) and automatically logs them out, clearing the AES key from the session. This is a standard feature in all real password managers and shows faculty that you think about physical security, not just cryptographic security.

Step-by-Step Implementation

Step 1 — Track last activity in the session

In app.py, add this function before your routes and call it at the start of each protected route:

```
from datetime import datetime, timedelta

AUTO_LOCK_MINUTES = 5   # Lock after 5 minutes of inactivity

def check_session_timeout():
    """Returns True if session is still valid, False if timed out."""
    if "aes_key" not in session:
        return False
    last_active = session.get("last_active")
    if not last_active:
        session["last_active"] = datetime.utcnow().isoformat()
        return True
    last_dt = datetime.fromisoformat(last_active)
    if datetime.utcnow() - last_dt > timedelta(minutes=AUTO_LOCK_MINUTES):
        session.clear()   # Lock the vault
        return False
    # Still active — update timestamp
    session["last_active"] = datetime.utcnow().isoformat()
    return True
```

Step 2 — Apply timeout check to the vault route

```
# At the top of the vault() function, add:
@app.route("/vault")
def vault():
    if not check_session_timeout():
        flash("Session timed out. Please log in again.", "error")
        return redirect(url_for("login"))
    # ... rest of function unchanged ...
```

Step 3 — Add client-side warning countdown

Add this JavaScript to vault.html just before the closing script tag. It shows a warning 60 seconds before auto-lock and then forces a logout:

```
const AUTO_LOCK_MS  = 5 * 60 * 1000;   // 5 minutes in ms
const WARN_BEFORE   = 60 * 1000;         // warn 60s before lock
let   lockTimer, warnTimer;

function resetLockTimers() {
```

```
  clearTimeout(lockTimer);
  clearTimeout(warnTimer);
  warnTimer = setTimeout(() => {
    showToast("Vault locks in 60 seconds due to inactivity!", "error");
  }, AUTO_LOCK_MS - WARN_BEFORE);
  lockTimer = setTimeout(() => {
    window.location.href = "/logout";
  }, AUTO_LOCK_MS);
}

// Reset timers on any user interaction
["mousemove", "keydown", "click", "scroll"].forEach(event => {
  document.addEventListener(event, resetLockTimers, { passive: true });
});

resetLockTimers();  // Start on page load
```

> **NOTE: Faculty Talking Point**
> "VaultX implements both server-side and client-side session timeout. The server checks the last-active timestamp on every request, so even if JavaScript is disabled, the session still expires. The client-side countdown gives the user a 60-second warning before forced logout. This is the same approach used by banking applications."

FEATURE 03 **Password History & Versioning**

Difficulty: Easy-Medium Impact: HIGH Est. Time: 1 hour

What It Is

Every time a password is updated, the old version is saved to a history table (encrypted). Users can view their last 5 passwords for any site and restore a previous one if needed. This prevents accidental overwrites and mimics a feature found in enterprise password managers.

Step-by-Step Implementation

Step 1 — Add a history table to vault.py

Inside the _init_db() method of the Vault class, add a second CREATE TABLE statement:

```
conn.execute("""
    CREATE TABLE IF NOT EXISTS password_history (
        id                  INTEGER PRIMARY KEY AUTOINCREMENT,
        entry_id            INTEGER NOT NULL,
        site                TEXT    NOT NULL,
        password_ciphertext  TEXT    NOT NULL,
        password_nonce       TEXT    NOT NULL,
        changed_at          TEXT    DEFAULT (datetime("now"))
    )
""")
```

Step 2 — Save old password before updating

In vault.py, modify the update_password() method to first copy the old password to history, then perform the update:

```
def update_password(self, entry_id, site, username, password, category,
notes):
    # Step 1: Save current password to history
    with sqlite3.connect(self.db_path) as conn:
        old = conn.execute(
            "SELECT password ciphertext, password nonce FROM passwords WHERE
id=?",
            (entry_id,)
        ).fetchone()
        if old:
            conn.execute(
                "INSERT INTO password_history (entry_id, site,
password_ciphertext, password_nonce)"
                " VALUES (?, ?, ?, ?)",
                (entry_id, site, old[0], old[1])
            )
            # Keep only last 5 versions
            conn.execute(
                "DELETE FROM password_history WHERE entry_id=? AND id NOT IN"
                " (SELECT id FROM password_history WHERE entry_id=?"
                "  ORDER BY changed_at DESC LIMIT 5)",
                (entry_id, entry_id)
            )
    # Step 2: Perform the actual update (existing code below)
    enc_pw = encrypt(self.key, password)
    # ... rest of existing update code ...
```

Step 3 — Add a route to view history

In app.py, add this route:

```
@app.route("/vault/history/<int:entry_id>")
def password_history(entry_id):
    if "aes_key" not in session:
        return jsonify({"error": "Not authenticated"}), 401
    key = bytes.fromhex(session["aes_key"])
    from crypto import decrypt
    with __import__("sqlite3").connect("vault.db") as conn:
        conn.row_factory = __import__("sqlite3").Row
        rows = conn.execute(
            "SELECT * FROM password_history WHERE entry_id=? ORDER BY
changed_at DESC",
            (entry_id,)
        ).fetchall()
    history = []
    for row in rows:
        row = dict(row)
        row["password"] = decrypt(key, row["password_nonce"],
row["password_ciphertext"])
        row.pop("password_ciphertext"); row.pop("password_nonce")
        history.append(row)
    return jsonify(history)
```

FEATURE 04 **Vault Health Dashboard**

Difficulty: Medium Impact: MAXIMUM Est. Time: 2 hours

What It Is

A dedicated page that analyzes every password in the vault and shows: how many are weak, how many are reused across multiple sites, how many are older than 90 days, and breach check results. This is the most visually impressive feature for a faculty demo — faculty can instantly see value without understanding the code.

What the Dashboard Shows

- Security Score — an overall percentage score (0–100%) based on password strength averages
- Weak Passwords — list of every entry where strength score is below 50%
- Reused Passwords — entries where the same password is used on multiple sites
- Old Passwords — entries that have not been changed in more than 90 days
- Breach-Check Summary — run HIBP check on all passwords in one click
- Strength Distribution Chart — bar chart showing how many passwords fall in each strength tier

Step-by-Step Implementation

Step 1 — Add a health analysis method to vault.py

```
def analyze_health(self) -> dict:
    """Decrypt all passwords and analyze for weakness/reuse/age."""
    from utils import check_strength
    from datetime import datetime, timedelta
    from crypto import decrypt

    with sqlite3.connect(self.db_path) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute("SELECT * FROM passwords").fetchall()

    weak, reuse_map, old = [], {}, []
    strength_counts = {"Very Weak": 0, "Weak": 0, "Fair": 0, "Strong": 0,
"Very Strong": 0}
    total_score = 0

    for row in rows:
        row = dict(row)
        pw  = decrypt(self.key, row["password_nonce"],
row["password_ciphertext"])
        s   = check_strength(pw)

        # Track strength
        strength_counts[s["label"]] = strength_counts.get(s["label"], 0) + 1
        total_score += s["percent"]

        # Weak passwords
        if s["percent"] < 50:
```

```
            weak.append({"id": row["id"], "site": row["site"], "strength": s})

        # Reuse detection (group by password hash, not plaintext)
        import hashlib
        pw_hash = hashlib.sha256(pw.encode()).hexdigest()[:16]
        reuse_map.setdefault(pw_hash, []).append({"id": row["id"], "site":
row["site"]})

        # Old passwords (> 90 days)
        updated = datetime.fromisoformat(row["updated_at"])
        if (datetime.utcnow() - updated).days > 90:
            old.append({"id": row["id"], "site": row["site"], "days":
(datetime.utcnow()-updated).days})

    reused = [group for group in reuse_map.values() if len(group) > 1]
    avg_score = round(total_score / len(rows)) if rows else 0

    return {
        "total"          : len(rows),
        "score"          : avg_score,
        "weak"           : weak,
        "reused"         : reused,
        "old"            : old,
        "strength counts": strength counts,
    }
```

Step 2 — Add dashboard route to app.py

```
@app.route("/vault/health")
def vault health():
    if "aes_key" not in session:
        return redirect(url_for("login"))
    v    = get_vault()
    data = v.analyze_health()
    return render_template("health.html", data=data)
```

Step 3 — Add a link in vault.html sidebar

In the sidebar section of vault.html, add this line inside .sidebar-bottom:

```
<a href="/vault/health" class="sidebar-action">□ Health Report</a>
```

> **□ TIP: Demo Strategy**
> During the faculty presentation, add 3–4 obviously weak passwords (like 'password123') before opening the Health Dashboard. The dashboard will immediately flag them as weak, making the feature very easy to understand visually without needing to explain the code.

FEATURE 05 **Chrome Browser Extension (Auto-Fill)**

Difficulty: Hard Impact: MAXIMUM Est. Time: 4–6 hours

What It Is

A Chrome extension that detects when you are on a login page, looks up the saved credentials for that site in VaultX, and fills in the username and password automatically. This is the single most useful feature of any password manager in real-world use, and showing it to faculty creates an immediate impression because they can see it working live.

How It Works

The Chrome extension is a small JavaScript program that runs inside Chrome. It communicates with your running Flask backend through a REST API. When you visit github.com, the extension calls http://127.0.0.1:5000/vault/lookup?site=github.com, receives the encrypted credentials, and injects them into the username and password fields on the page.

Step-by-Step Implementation

Step 1 — Create the extension folder

```
vaultx/
  extension/
    manifest.json       <- Extension config file
    popup.html          <- Small UI in the extension popup
    popup.js            <- JavaScript for the popup
    content.js          <- Injected into every webpage
    background.js       <- Runs in background (service worker)
    icon128.png         <- Extension icon (any 128x128 image)
```

Step 2 — Create manifest.json

```
{
  "manifest_version": 3,
  "name": "VaultX Auto-Fill",
  "version": "1.0",
  "description": "Auto-fill passwords from your VaultX vault.",
  "permissions": ["activeTab", "scripting"],
  "host_permissions": ["http://127.0.0.1:5000/*"],
  "action": {
    "default_popup": "popup.html",
    "default_icon": "icon128.png"
  },
  "content_scripts": [{
    "matches": ["<all_urls>"],
    "js": ["content.js"]
  }]
}
```

Step 3 — Create popup.js

```
// Runs when user clicks the VaultX icon in Chrome toolbar
document.addEventListener("DOMContentLoaded", async () => {
  const [tab]    = await chrome.tabs.query({ active: true, currentWindow: true
});
  const url      = new URL(tab.url);
```

```
  const site     = url.hostname.replace("www.", "");

  // Ask Flask backend for matching credentials
  const res  = await fetch(`http://127.0.0.1:5000/vault/lookup?site=${site}`);
  const data = await res.json();

  if (data.error) {
    document.getElementById("status").textContent = "No saved password for " +
site;
    return;
  }

  document.getElementById("site").textContent     = site;
  document.getElementById("username").textContent = data.username;

  document.getElementById("fillBtn").onclick = async () => {
    chrome.scripting.executeScript({
      target: { tabId: tab.id },
      func: (user, pw) => {
        const inputs = document.querySelectorAll("input");
        inputs.forEach(el => {
          if (el.type === "email" || el.name?.includes("user") ||
el.name?.includes("email"))
            el.value = user;
          if (el.type === "password")
            el.value = pw;
        });
      },
      args: [data.username, data.password]
    });
  };
});
```

Step 4 — Add lookup route to app.py

```
@app.route("/vault/lookup")
def lookup():
    if "aes_key" not in session:
        return jsonify({"error": "Not logged in"}), 401
    site  = request.args.get("site", "").lower().strip()
    v     = get_vault()
    with __import__("sqlite3").connect("vault.db") as conn:
        conn.row_factory = __import__("sqlite3").Row
        row = conn.execute(
            "SELECT * FROM passwords WHERE LOWER(site) LIKE ? LIMIT 1",
            (f"%{site}%",)
        ).fetchone()
    if not row:
        return jsonify({"error": "Not found"})
    from crypto import decrypt
    return jsonify({
        "username": row["username"],
        "password": decrypt(bytes.fromhex(session["aes_key"]),
                            row["password nonce"],
row["password_ciphertext"])
    })
```

Step 5 — Load the extension in Chrome

1. Open Chrome and go to: chrome://extensions
2. Turn on 'Developer mode' using the toggle in the top-right corner.
3. Click 'Load unpacked' and select the extension/ folder.
4. The VaultX icon will appear in your Chrome toolbar.
5. Make sure the Flask app is running, then visit any site — click the icon to auto-fill.

FEATURE 06 **Argon2id (Upgrade from PBKDF2)**

Difficulty: Easy Impact: MAXIMUM KNOWLEDGE SIGNAL Est. Time: 30 minutes

What It Is and Why It Matters

Argon2id won the Password Hashing Competition (PHC) in 2015 and is now the recommended algorithm by OWASP (Open Web Application Security Project) and NIST for key derivation. It is better than PBKDF2 in one critical way: it is deliberately memory-hard, meaning an attacker needs large amounts of RAM to run brute-force attacks. PBKDF2 is only time-hard (slow), but GPUs can run thousands of PBKDF2 operations in parallel cheaply. Argon2id forces each attempt to consume 64MB of RAM, making GPU-based attacks 100x more expensive.

Replacing PBKDF2 with Argon2id in the code takes about 30 minutes and immediately shows faculty that you know about the state of the art in cryptographic research, not just textbook algorithms.

Step-by-Step Implementation

Step 1 — Install argon2-cffi

```
pip install argon2-cffi
```

Step 2 — Replace the derive_key function in crypto.py

Find the derive_key function in crypto.py and replace it entirely with the following:

```
from argon2.low_level import hash_secret_raw, Type

# OWASP recommended parameters for Argon2id (2024):
ARGON2_TIME_COST   = 3        # Number of iterations
ARGON2_MEM_COST    = 65536    # 64 MB of RAM per attempt
ARGON2_PARALLELISM = 4        # 4 parallel threads

def derive_key(password: str) -> bytes:
    """
    Derive a 256-bit AES key using Argon2id.
    Argon2id = memory-hard + time-hard.
    GPU attacks are 100x more expensive than with PBKDF2.
    OWASP recommends Argon2id over PBKDF2 as of 2023.
    """
    salt = _get_or_create_salt()
    key  = hash_secret_raw(
        secret      = password.encode("utf-8"),
        salt        = salt,
```

```
        time_cost   = ARGON2_TIME_COST,
        memory_cost = ARGON2_MEM_COST,
        parallelism = ARGON2_PARALLELISM,
        hash_len    = 32,          # 32 bytes = 256-bit key
        type        = Type.ID,     # ID = Argon2id variant
    )
    return key
```

KEY PHRASE FOR FACULTY

"PBKDF2 is time-hard: it makes each guess slow, but attackers can run thousands of guesses in parallel on a GPU very cheaply. Argon2id is memory-hard: each guess requires 64 megabytes of RAM. A GPU with 8GB of RAM can only run 128 guesses simultaneously instead of thousands. This directly increases the cost of an attack by a factor of 100 or more. Argon2id won the Password Hashing Competition in 2015 and is the current OWASP recommendation."

FEATURE 07 **Encrypted Secure Notes**

Difficulty: Easy Impact: MEDIUM Est. Time: 45 minutes

What It Is

A separate section in the vault where users can store text notes that are AES-256-GCM encrypted, just like passwords. Useful for storing recovery codes, API keys, Wi-Fi passwords, software license keys, and any sensitive text. The existing crypto.py already handles encryption — you just need a new database table and a few new routes.

Step-by-Step Implementation

Step 1 — Add a notes table in vault.py _init_db()

```
conn.execute("""
    CREATE TABLE IF NOT EXISTS secure_notes (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        title       TEXT    NOT NULL,
        content_ciphertext TEXT NOT NULL,
        content_nonce      TEXT NOT NULL,
        category    TEXT    DEFAULT "General",
        created_at  TEXT    DEFAULT (datetime("now")),
        updated_at  TEXT    DEFAULT (datetime("now"))
    )
""")
```

Step 2 — Add note methods to the Vault class

```
def add_note(self, title: str, content: str, category: str = "General") ->
int:
    enc = encrypt(self.key, content)
    with sqlite3.connect(self.db_path) as conn:
        cur = conn.execute(
```

```
            "INSERT INTO secure_notes (title, content_ciphertext,
content_nonce, category)"
            " VALUES (?, ?, ?, ?)",
            (title, enc["ciphertext"], enc["nonce"], category)
        )
        return cur.lastrowid

def get_note(self, note_id: int) -> dict | None:
    with sqlite3.connect(self.db_path) as conn:
        conn.row_factory = sqlite3.Row
        row = conn.execute("SELECT * FROM secure_notes WHERE id=?",
(note_id,)).fetchone()
    if not row:
        return None
    row = dict(row)
    row["content"] = decrypt(self.key, row["content_nonce"],
row["content_ciphertext"])
    row.pop("content_ciphertext"); row.pop("content_nonce")
    return row

def list_notes(self) -> list:
    with sqlite3.connect(self.db_path) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute(
            "SELECT id, title, category, created_at FROM secure_notes ORDER BY
title"
        ).fetchall()
    return [dict(r) for r in rows]
```

Step 3 — Add routes in app.py

```
@app.route("/notes")
def notes():
    if "aes_key" not in session: return redirect(url_for("login"))
    v = get_vault()
    return render_template("notes.html", notes=v.list_notes())

@app.route("/notes/add", methods=["POST"])
def add_note():
    if "aes_key" not in session: return jsonify({"error": "Not
authenticated"}), 401
    d  = request.get_json()
    v  = get_vault()
    id = v.add_note(d["title"], d["content"], d.get("category", "General"))
    return jsonify({"success": True, "id": id})
```

FEATURE 08 **Import Passwords from Chrome / Firefox**

Difficulty: Medium Impact: HIGH Est. Time: 1.5 hours

What It Is

Chrome and Firefox can export saved passwords as a CSV file. VaultX can read that CSV file, parse the site/username/password columns, encrypt each entry, and import them all into the vault in one step.

This makes VaultX immediately useful to anyone who has been saving passwords in their browser — which is almost everyone.

Step-by-Step Implementation

Step 1 — How to export passwords from Chrome

6. Open Chrome and go to: chrome://password-manager/passwords
7. Click the Settings icon (⚙) → Export passwords.
8. Save the .csv file to your computer.

The CSV format is: name, url, username, password (with a header row)

Step 2 — Add import method to vault.py

```
import csv, io

def import_from_csv(self, csv_text: str) -> dict:
    """
    Parse Chrome/Firefox password CSV and import all entries.
    Chrome format: name,url,username,password
    Firefox format: url,username,password,httpRealm,...
    Returns: {"imported": N, "skipped": N, "errors": []}
    """
    reader   = csv.DictReader(io.StringIO(csv_text))
    imported = skipped = 0
    errors   = []

    for i, row in enumerate(reader):",
        try:
            # Support both Chrome and Firefox column names
            site = (row.get("name") or row.get("url") or "").strip()
            user = (row.get("username") or "").strip()
            pw   = (row.get("password") or "").strip()

            if not site or not pw:
                skipped += 1; continue

            # Clean up URL to just domain name
            if site.startswith("http"):
                from urllib.parse import urlparse
                site = urlparse(site).netloc.replace("www.", "")

            self.add_password(site, user, pw, "Imported")
            imported += 1
        except Exception as e:
            errors.append(f"Row {i}: {e}")

    return {"imported": imported, "skipped": skipped, "errors": errors}
```

Step 3 — Add import route to app.py

```
@app.route("/vault/import", methods=["POST"])
def import_csv():
    if "aes_key" not in session:
        return jsonify({"error": "Not authenticated"}), 401
    if "file" not in request.files:
        return jsonify({"error": "No file provided"}), 400
```

```
    f        = request.files["file"]
    csv_text = f.read().decode("utf-8")
    v        = get_vault()
    result   = v.import_from_csv(csv_text)
    return jsonify(result)
```

FEATURE 09 **Login Attempt Limiting (Anti-Brute-Force)**

Difficulty: Easy Impact: HIGH Est. Time: 30 minutes

What It Is

After a set number of consecutive failed login attempts (e.g. 5), the app locks further attempts for a cooldown period (e.g. 15 minutes). This directly implements protection against automated brute-force attacks at the application layer, in addition to bcrypt's natural slowness.

Step-by-Step Implementation

Add these lines to app.py, before the login route:

```
from collections import defaultdict
from datetime import datetime, timedelta

login_attempts  = defaultdict(int)       # ip -> attempt count
login_lockouts  = {}                      # ip -> lockout-until datetime

MAX_ATTEMPTS    = 5
LOCKOUT_MINUTES = 15

def get client ip():
    return request.remote_addr or "unknown"

def is_locked_out(ip: str) -> bool:
    if ip not in login_lockouts:
        return False
    if datetime.utcnow() < login_lockouts[ip]:
        return True
    # Lockout expired — reset
    del login_lockouts[ip]
    login attempts[ip] = 0
    return False

def record_failed_attempt(ip: str):
    login_attempts[ip] += 1
    if login_attempts[ip] >= MAX_ATTEMPTS:
        login_lockouts[ip] = datetime.utcnow() +
timedelta(minutes=LOCKOUT_MINUTES)
        login_attempts[ip] = 0
        return True   # Just got locked out
    return False
```

Then at the top of the login POST handler, add:

```
ip = get_client_ip()
if is_locked_out(ip):
    until = login_lockouts[ip].strftime('%H:%M:%S')
    flash(f'Too many failed attempts. Try again after {until} UTC.', 'error')
    return render_template('login.html')

# ... existing verify_master_password check ...
if verify_master_password(password):
    login_attempts[ip] = 0    # Reset on success
    # ... rest of login ...
else:
    locked = record_failed_attempt(ip)
    if locked:
        flash(f'Too many attempts. Locked for {LOCKOUT_MINUTES} minutes.',
'error')
    else:
        remaining = MAX_ATTEMPTS - login_attempts[ip]
        flash(f'Incorrect password. {remaining} attempt(s) remaining.',
'error')
```

FEATURE 10 **Password Expiry Reminders**

Difficulty: Easy Impact: MEDIUM Est. Time: 45 minutes

What It Is

Shows a warning badge on old passwords that have not been changed in more than 90 days. Security best practice recommends rotating critical passwords (banking, email) periodically. This feature makes VaultX proactively helpful rather than just a storage tool.

Step-by-Step Implementation

Step 1 — Add expiry column to passwords table

```
# In vault.py _init_db(), after the existing CREATE TABLE, run a migration:
try:
    conn.execute("ALTER TABLE passwords ADD COLUMN expiry_days INTEGER DEFAULT
90")
except sqlite3.OperationalError:
    pass  # Column already exists
```

Step 2 — Calculate expiry in list_all()

Modify the list_all() method to include expiry information:

```
from datetime import datetime

def list_all(self) -> list[dict]:
    with sqlite3.connect(self.db_path) as conn:
```

```
        conn.row_factory = sqlite3.Row
        rows = conn.execute(
            "SELECT id, site, username, category, expiry_days, created_at,
updated_at"
            " FROM passwords ORDER BY site COLLATE NOCASE"
        ).fetchall()
    result = []
    for r in [dict(row) for row in rows]:
        updated  = datetime.fromisoformat(r["updated_at"])
        age_days = (datetime.utcnow() - updated).days
        exp_days = r.get("expiry_days", 90) or 90
        r["age_days"]  = age_days
        r["is_expired"] = age_days > exp_days
        r["days_until_expiry"] = max(0, exp_days - age_days)
        result.append(r)
    return result
```

Step 3 — Show expiry badge in vault.html

In the site name cell of the password table (the td-site column), add this inline Jinja2 check:

```
{% if e.is_expired %}
  <span class='expiry-badge'>⚠ Expired</span>
{% elif e.days_until_expiry <= 7 %}
  <span class='expiry-badge expiry-warn'>{{ e.days_until_expiry }}d
left</span>
{% endif %}
```

FEATURE 11 **Fuzzy Search**

Difficulty: Easy Impact: MEDIUM Est. Time: 30 minutes

What It Is

Standard search only matches if you type the exact site name. Fuzzy search also matches typos and partial names — typing 'githb' still finds 'github.com'. It uses the Levenshtein distance algorithm (edit distance) to measure how similar two strings are.

Step-by-Step Implementation

Step 1 — Install rapidfuzz

```
pip install rapidfuzz
```

Step 2 — Add fuzzy search route to app.py

```
from rapidfuzz import fuzz

@app.route("/vault/search")
def fuzzy_search():
    if "aes_key" not in session:
```

```
        return jsonify({"error": "Not authenticated"}), 401
    query   = request.args.get("q", "").lower().strip()
    if not query:
        return jsonify([])
    v       = get_vault()
    entries = v.list_all()
    results = []
    for e in entries:
        score = fuzz.partial_ratio(query, e["site"].lower())
        if score >= 60:    # 60% similarity threshold
            e["match_score"] = score
            results.append(e)
    results.sort(key=lambda x: x["match_score"], reverse=True)
    return jsonify(results[:10])  # Return top 10 matches
```

Step 3 — Update search input in vault.html

Replace the filterSearch JavaScript function with a debounced version that calls the fuzzy search API:

```
let searchDebounce;
function filterSearch(val) {
  clearTimeout(searchDebounce);
  if (!val.trim()) { filterTable(); return; }
  searchDebounce = setTimeout(async () => {
    const res     = await fetch(`/vault/search?q=${encodeURIComponent(val)}`);
    const results = await res.json();
    const ids     = new Set(results.map(r => r.id));
    document.querySelectorAll(".pw-row").forEach(row => {
      row.style.display = ids.has(parseInt(row.dataset.id)) ? "" : "none";
    });
  }, 250);  // Wait 250ms after user stops typing
}
```

FEATURE 12 **Windows Desktop App with CustomTkinter**

Difficulty: Medium Impact: HIGH Est. Time: 3–4 hours

What It Is

A native Windows desktop application using CustomTkinter — a modern-looking Python GUI library. Instead of opening a browser, users get a proper Windows app with a taskbar icon that opens directly into the vault. CustomTkinter provides a sleek dark-mode UI that looks like a modern application, not a 1990s Tkinter widget.

Step-by-Step Implementation

Step 1 — Install CustomTkinter

```
pip install customtkinter
```

Step 2 — Create desktop_app.py

Create a new file desktop_app.py in the vaultx folder:

```
import customtkinter as ctk
from auth   import verify_master_password, master_password_exists,
set_master_password
from crypto import derive_key
from vault  import Vault
from utils  import generate_password, check_strength

ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

DB = "vault desktop.db"

class VaultXApp(ctk.CTk):
    def   init  (self):
        super().__init__()
        self.title("VaultX — Secure Password Manager")
        self.geometry("900x600")
        self.minsize(700, 500)
        self.aes_key = None
        self.vault   = None
        if not master_password_exists():
            self.show_setup()
        else:
            self.show_login()

    def show_login(self):
        self.clear()
        frame = ctk.CTkFrame(self, width=400)
        frame.place(relx=0.5, rely=0.5, anchor="center")
        ctk.CTkLabel(frame, text="□ VaultX", font=("Arial", 28,
"bold")).pack(pady=(30,10))
        ctk.CTkLabel(frame, text="Enter master password").pack()
        pw_entry = ctk.CTkEntry(frame, show="*", width=300)
        pw_entry.pack(pady=10)
        pw_entry.focus()
        msg = ctk.CTkLabel(frame, text="", text_color="red")
        msg.pack()
        def attempt():
            pw = pw_entry.get()
            if verify_master_password(pw):
                self.aes_key = derive_key(pw)
                self.vault   = Vault(DB, self.aes_key)
                pw = None
                self.show_vault()
            else:
                msg.configure(text="Incorrect password")
                pw_entry.delete(0, "end")
        ctk.CTkButton(frame, text="Unlock Vault",
command=attempt).pack(pady=10)
        pw_entry.bind("<Return>", lambda e: attempt())

    def show vault(self):
        self.clear()
        entries = self.vault.list_all()
        # Sidebar
        sidebar = ctk.CTkFrame(self, width=180)
        sidebar.pack(side="left", fill="y", padx=(10,0), pady=10)
```

```
        ctk.CTkLabel(sidebar, text="□ VaultX", font=("Arial", 18,
"bold")).pack(pady=15)
        ctk.CTkButton(sidebar, text="+ Add Password",
command=self.open_add).pack(pady=5, padx=10)
        ctk.CTkButton(sidebar, text="⚡ Generator",
command=self.open_gen).pack(pady=5, padx=10)
        ctk.CTkButton(sidebar, text="□ Lock",
                      fg_color="transparent",
command=self.lock).pack(side="bottom", pady=15, padx=10)
        # Main table
        main = ctk.CTkScrollableFrame(self)
        main.pack(side="right", fill="both", expand=True, padx=10, pady=10)
        for e in entries:
            row = ctk.CTkFrame(main)
            row.pack(fill="x", pady=3)
            ctk.CTkLabel(row, text=e["site"],     width=200,
anchor="w").pack(side="left", padx=10)
            ctk.CTkLabel(row, text=e["username"] or "—", width=200,
anchor="w").pack(side="left")
            ctk.CTkLabel(row, text=e["category"], width=120,
anchor="w").pack(side="left")
            eid = e["id"]
            ctk.CTkButton(row, text="Copy", width=60,
                          command=lambda i=eid:
self.copy_pw(i)).pack(side="right", padx=5)
            ctk.CTkButton(row, text="Delete", width=60, fg_color="#8B1A1A",
                          command=lambda i=eid: self.delete_entry(i,
main)).pack(side="right", padx=2)

    def copy_pw(self, entry_id):
        entry = self.vault.get_full_entry(entry_id)
        if entry:
            self.clipboard_clear()
            self.clipboard_append(entry["password"])
            self.after(30000, self.clipboard_clear)  # Clear in 30s

    def delete_entry(self, entry_id, container):
        if ctk.CTkInputDialog(text="Type YES to confirm:",
title="Delete?").get_input() == "YES":
            self.vault.delete_password(entry_id)
            self.show_vault()

    def open_gen(self):
        win = ctk.CTkToplevel(self)
        win.title("Password Generator"); win.geometry("400x300")
        pw_var = ctk.StringVar()
        ctk.CTkLabel(win, textvariable=pw_var, font=("Courier New",
14)).pack(pady=20)
        def gen():
            pw = generate_password(16, True, True, True)
            pw_var.set(pw)
        def copy():
            self.clipboard_clear(); self.clipboard_append(pw_var.get())
        gen()
        ctk.CTkButton(win, text="Regenerate", command=gen).pack(pady=5)
        ctk.CTkButton(win, text="Copy",       command=copy).pack(pady=5)

    def open_add(self):
        win = ctk.CTkToplevel(self)
        win.title("Add Password"); win.geometry("400x380")
        fields = {}
```

```
        for label, key, show in [("Site","site",""),
("Username","username",""), ("Password","password","*")]:
            ctk.CTkLabel(win, text=label).pack(pady=(10,2))
            e = ctk.CTkEntry(win, show=show, width=300)
            e.pack(); fields[key] = e
        def save():
            self.vault.add_password(fields["site"].get(),
fields["username"].get(), fields["password"].get())
            win.destroy(); self.show_vault()
        ctk.CTkButton(win, text="Save", command=save).pack(pady=15)

    def lock(self):
        self.aes_key = None; self.vault = None
        self.show_login()

    def clear(self):
        for w in self.winfo_children(): w.destroy()

    def show_setup(self):
        self.clear()
        frame = ctk.CTkFrame(self, width=400)
        frame.place(relx=0.5, rely=0.5, anchor="center")
        ctk.CTkLabel(frame, text="□ VaultX Setup", font=("Arial", 24,
"bold")).pack(pady=20)
        pw1 = ctk.CTkEntry(frame, show="*", placeholder_text="Master
password", width=300); pw1.pack(pady=8)
        pw2 = ctk.CTkEntry(frame, show="*", placeholder_text="Confirm
password", width=300); pw2.pack(pady=8)
        msg = ctk.CTkLabel(frame, text="", text_color="red"); msg.pack()
        def create():
            p1, p2 = pw1.get(), pw2.get()
            if len(p1) < 8: msg.configure(text="At least 8 characters");
return
            if p1 != p2:    msg.configure(text="Passwords do not match");
return
            set_master_password(p1); self.show_login()
        ctk.CTkButton(frame, text="Create Vault",
command=create).pack(pady=15)

if __name__ == "__main__":
    app = VaultXApp()
    app.mainloop()
```

Step 3 — Run the desktop app

```
python desktop_app.py
```

Step 4 — Package as a .exe (optional but impressive)

```
pip install pyinstaller
pyinstaller --onefile --windowed --name VaultX desktop_app.py
# Output: dist/VaultX.exe  (share this file, Python not required)
```

Summary — What to Say for Each Feature

This table gives the one-sentence faculty-facing explanation for each feature. Memorize the ones that match the features you implement.

Feature	One-Sentence Explanation for Faculty
TOTP 2FA	Even if the master password is stolen, the vault cannot be opened without the 6-digit rotating code from the user's phone — same standard as Gmail 2FA.
Auto-Lock	The vault automatically locks after 5 minutes of inactivity, both server-side and client-side, matching the behavior of banking applications.
Password History	Every time a password is changed, the encrypted previous version is saved to a history table — no credential is ever permanently lost.
Health Dashboard	A visual analytics page that flags weak, reused, and expired passwords across the entire vault — the same feature that drives premium subscriptions in commercial managers.
Browser Extension	The Chrome extension detects the current site, queries the Flask REST API, and injects the username and password into the login form — demonstrating full-stack integration.
Argon2id	Argon2id won the Password Hashing Competition in 2015 and is the current OWASP recommendation over PBKDF2 — it is memory-hard, making GPU attacks 100x more expensive.
Secure Notes	A separate encrypted section for recovery codes, API keys, and license keys — all encrypted with the same AES-256-GCM as passwords.
CSV Import	Users can export passwords from Chrome or Firefox and import them all in one step, making VaultX immediately practical for real-world use.
Login Limiting	After 5 consecutive failed login attempts, the application enforces a 15-minute lockout — direct defense against automated brute-force attacks at the application layer.
Expiry Reminders	Entries older than 90 days are flagged with an expiry warning, encouraging proactive password rotation for critical accounts.
Fuzzy Search	Uses the Levenshtein distance algorithm to find passwords even with typos — typing 'githb' still finds 'github.com'.
Desktop App	A native Windows application built with CustomTkinter, packaged as a standalone .exe using PyInstaller — no browser or Python installation required on the end user's machine.